ELTE IK - Computer Science BSc

Final Exam Topics

7. Algorithmic patterns

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Algorithmic patterns

The concept of the enumerator. Enumerators of well-known collections (interval, array, sequence, sequential input file). Algorithmic patterns formulated on enumerators (summation, counting, maximum selection, conditional maximum search, linear search, selection). The method of reduction. Testing programs created with algorithmic patterns.

1 Steps of solving a simple programming task

1.1 Introduction

Solving a programming task contains quite a few activities beyond coding. The first thing to do is the precise definition of the task, the specification. Besides the textual and formalized, mathematical description of the task (the so-called narrower interpretation of the specification), this also contains the requirements imposed on the solution and the environmental needs (which is the so-called broader interpretation of the specification).

Based on the specification one can design the program; the algorithm of the solution and the description of the data used by the algorithm can be produced. The refinement of the algorithm and the data structure proceeds in parallel, all the way down to the level that the programmer, based on their knowledge, can already code easily and error-free. It often happens that gaps in the specification come to light during design, so here we can expect to step back.

After writing the algorithm, coding can follow. If the task setter did not fix it, then before this we can choose a programming language for the solution. The result of coding is the program written in the programming language.

In its first version the program is generally never flawless; we can speak of its correctness only once we have convinced ourselves of it. One possible method of examining correctness is testing. During this we try out the program with test data, and from the result given for these we draw conclusions about correctness. (Let us have no illusions that the correctness of a program can be decided by testing. For whether the program is actually correct – unfortunately – does not follow from the fact that we did not find an error.)

If during testing we encounter an error symptom, then debugging can follow: finding the statement causing the error symptom, and then fixing the error. The fixing of the error can reach back into several phases. It is conceivable that we have to fix a coding error, but it may also be that we already made the error during design. After fixing we have to test again, since – let us be honest with ourselves! – it is not excluded that we fix incorrectly, or – with mild optimism we assert – that the fix reveals new errors, ...

The end result of this process is the correct program. With this, however, the production of the program is by no means finished. Now come the quality requirements. On the one hand we have to examine efficiency (execution

time, storage usage), on the other hand convenient usability. Here we can again step back into the coding or the design phase. With this we have arrived at the good program.

1.2 Specification

The first step of the program-making process is the definition of the task, its precise "reformulation". What should it be like, what should we expect from it? Let us look at a few – seemingly good – requirements, for now as keywords! (In what follows we will discuss the narrower interpretation of the specification.) The specification should be:

- correct, unambiguous, precise, complete
- short, concise, which is most easily achieved by building it on known formalisms
- illustrative, understandable (which formalization sometimes makes more difficult)

As a first approximation the specification could be the text of the tasks. This, however, can raise several problems:

- on what basis do we give the solution
- what exactly has to be specified?

For example, the task of giving the tallest among N people. What does giving the tallest person mean? Do we give their sequence number, or their name, or their personal ID number, or their height, possibly all of these? As a lesson we can conclude that the specification has to contain the description of the input and output data.

```
Input:
    N : the number of people,
    A : the sequence containing their heights.

Output:
    MAX : the sequence number of the tallest person.
```

Do we know what values the input and output variables can take? For example, in what unit of measurement should the heights of the people be given? What value can the resulting sequence number be: do we number from 1, or from 0? We can therefore conclude that in the specification we also have to give the value set of the input and output variables.

```
Input:
    N : the number of people, a natural number;
    A : the sequence containing their heights, integers expressing the height
        in centimeters (the sequence is indexed from 1 to N).
Output:
    MAX : the sequence number of the tallest person, a natural number between 1 and N.
```

Now we have precisely determined the value set of the input and output variables; the only problem is that we have not defined the concepts used in the task and the rule for computing the results. The specification therefore has to contain the definition of the concepts used in the task, as well as the rule for computing the result. Here one could also give the relationships relating to the input data. The conditions imposed on the input and on the output data are called the precondition and the postcondition, respectively. The precondition is very often an identically true statement, that is, we do not restrict the value set of the input data with any "extra" condition.

```
Input:
     N : the number of people, a natural number,
     A : the sequence containing their heights, integers,
         which contain the height in centimeters (the sequence is indexed from 1 to N).
Output:
     MAX : the sequence number of the tallest person, a natural number between 1 and N.

Precondition:
    the A[i] are positive.

Postcondition:
    MAX is a number between 1 and N for which A[MAX] is greater than or equal
    to any element of the sequence (between the 1st and the Nth).
```

A further problem can arise in connection with any task: based on the above, we can also produce a solution significantly different from the "expected" – we can safely say: "banal" – one that conforms to the pre- and postcondition.

Here, of course, this is about the tacit (that is, not yet formulated, not spoken) assumption that the value of the input variables does not change. This, unfortunately, is not necessarily true. To solve the problem we can follow two paths (in what follows we will apply both):

- we automatically include in the postcondition that "and the value of the input variables does not change", and separately highlight it if this is nevertheless not so;
- we formulate the pre- and postcondition for the parameters of the program, which we formally distinguish from the variables of the program, and because of this it will not be the parameters that change but the program variables (in this case, of course, in the pre- and postcondition we have to formulate the identity of the values of the parameters and the corresponding program variables).

From the second solution it follows that we have to distinguish from each other the pre- and postcondition of the task and of the program! This makes the formulation of the task lengthier – though more precise –, because of which we will apply it less often.

It can happen that, based on the formulation of the task, the result cannot be unambiguously determined, since several solutions conforming to the postcondition exist. This phenomenon is the so-called nondeterminism of the task. For this nondeterministic task we therefore have to write a deterministic program, whose postcondition can no longer permit ambiguity, nondeterminism. Because of this problem we therefore in any case have to distinguish from each other the pre- and postcondition of the task and of the program!

```
Input:
    N : the number of people, a natural number,
    A : the sequence containing their heights, integers,
        which contain the height in centimeters (the sequence is indexed from 1 to N).
Output:
    MAX : the sequence number of the tallest person, a natural number between 1 and N.

Precondition:
   the A[i] are positive.

   Postcondition:
     MAX is a number between 1 and N for which A[MAX] is greater than or equal
    to any element of the sequence (between the 1st and the Nth).
```

```
Program postcondition:
    MAX is a number between 1 and N for which A[MAX] is greater than or equal
    to any element of the sequence (between the 1st and the Nth) and before it
    there is none equal to it.
```

We can conclude from this that the postcondition of the program can be stricter than that of the task, and in addition its precondition can be weaker.

Looking back at the "path traversed so far" by the specification, an illustrative model of task solving takes shape. Namely: we can safely say that the program solving the task defines an automaton whose momentary state is an element of the set "spanned" by the parameters of the task (the variables of the program). (This set has as many dimensions as the program has parameter variables; each dimension is the value set of one variable. So at a concrete moment in time the state of this "machine" is: the collection of the values of its variables valid at that moment.) We call this set the state space of the program. When we formulate the precondition, we essentially cut out from this state space the part (the subspace) from which, started, we can expect of our automaton (controlled by the solving program) that it produces the correct result in one of its final states. We designated the final state with the postcondition.

Accepting this model, one more question awaiting solution arises. Namely, when we write the program, at every turn we introduce more and more variables for storing partial results. The question arises: how can this be reconciled with the model just imagined? The answer is simple: each new variable fits a new dimension onto the state space created so far. So the process of programming – simplifying the matter – consists of nothing but making precise what the state space of the solving automaton should look like (and of course: how it has to get from one state to another). The "embryonic" state space determined by the parameters appearing in the task can be called the parameter space, which is only a subspace of the real state space of the program. This too suggests that the precondition of the task can be weaker (that is, the state set it designates) than the precondition of the program.

Let us now summarize what the parts of the specification are! These will be the above, as well as further restrictions relating to the program.

1. Specifying the task
    - the text of the task,
    - the naming of the input and output data, the description of their value set,
    - the definitions of the concepts used in the text of the task (using the concepts),
    - the precondition written for the input data (using the concepts),
    - the postcondition written for the output data.
2. Specifying the program
    - the naming of the input and output data, the description of their value set,
    - the program pre- and postcondition (possibly different from the pre- or postcondition of the task),
    - the definitions of the concepts used in the formulation of the task.

These are the elements of the abstract specification. The following are the parts of the secondary, we may say: technical specification:

- the description of the program's environment (computer, memory and peripheral requirement, programming language, necessary files, etc.),
- other requirements imposed on the program (quality, efficiency, portability, etc.).

The description without the technical specification is called the narrower specification of the program.

Program-style specification:

$A = (N : \mathbb{N}, A : \mathbb{N}^{1..N})$
$Pre = (\forall i \in 1..N : A_i > 0)$
$Post = (Pre \wedge \forall i \in 1..N : A_{MAX} >= A_i \wedge \forall j \in 1..MAX - 1 : A_i < A_{MAX})$

1.3 Design

During design we use algorithm-describing tools, the purpose of which is to describe the solution of tasks in a language independent of the programming language. Programming languages, namely, have strict syntax and contain frills irrelevant from the point of view of design. In the case of design in a programming language, rewriting the program to another language or another machine can become difficult.

Several kinds of algorithm-describing tools exist; in our studies we applied the structogram.

The structogram represents the program graph without edges. Thus a single basic element remains, the rectangle. With this basic element we can build up the usual structured basic constructs (and only those).

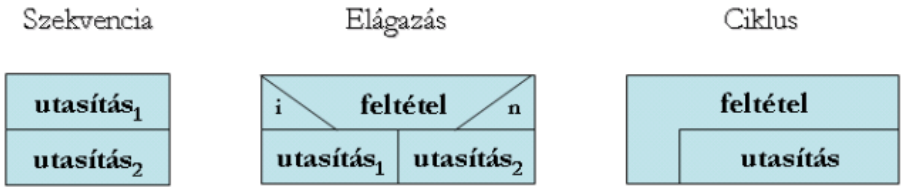


Figure 1: The composite basic constructs of the structogram.

For a sequence, the top-to-bottom order of the rectangles decides the order of execution. For the value true of the branch condition, the statement of the left rectangle marked with the letter i (igaz = true) has to be executed, and for the value false, that of the right rectangle marked with the letter n (nem = false). If one of the branches of the branching is empty, then the rectangle corresponding to it also remains empty. The loop is pre-tested, that is, the statement inside it has to be executed as long as the condition is true.

In place of the statements there can be a single elementary statement, one of the three algorithmic constructs, or a procedure call. This describing tool is usually extended with several more elements: the procedure definition, the multi-way branching, and the post-tested loop.

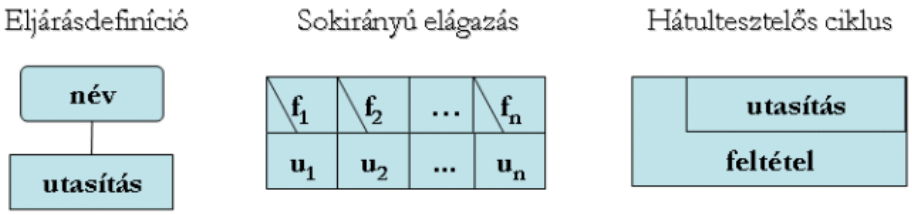


Figure 2: Further composite basic constructs of the structogram.

For a multi-way branching, the branch whose condition has the value true has to be executed (of these, in every case exactly one can hold).

The local data can be listed next to the rectangles of the procedures, after the procedure name.

Let us look at the following example described with this tool!

Task: Knowing the year-end averages of N students, give the number of students with an excellent average!

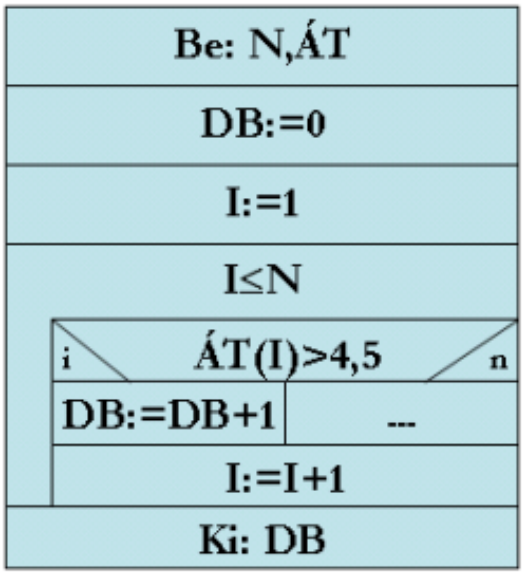


Figure 3: The solution of the example task with a structogram.

1.4 Implementation

A.k.a. coding.

1.5 Testing

The goal of testing is to find as many errors in the program as possible. To discover all errors, it seems obvious to try out the program with all possible input data. This, however, is unfortunately not possible.
As an example, consider the following – pseudocode-given – simple program:

```
Program:
    Variable A,B:Integer
    In: A,B
    Out: A/B
Program end.
```

Since we can store 2^{16} integers of distinct value, the total number of possibilities is 2^{32}, the writing of which already requires 9 digits. This would take a great deal of time, so it is not even a feasible path.

If we try out this program with input data in which A=0 or B=1, then the program works correctly and we cannot discover the error. After this we might think that we have ended up in a hopeless situation: for we cannot try out the program with every possible piece of data; and if we try it out with fewer, then it is possible that we do not notice the errors. The situation is not that bad: our goal can only be to carry out the testing with a method by which the number of trials can be strongly reduced.

A test case is the joint specification of the input and output data and conditions. We can say anything about the results of testing only if we have an idea of what result we expect for a given input.

Let us formulate the principles of testing:

- A good test case is one that with high probability reveals an as-yet-undiscovered error in the program. For example, a program computing the greatest common divisor of two numbers is completely unnecessary to try out with the pair [6,6] after the pair [5,5] (since the program could behave differently for [6,6] than for [5,5] only in the case of a very tricky, improbable typo).

- A test case consists not only of input data, but also of the results belonging to them. Otherwise we could not speak of the correct or erroneous nature of the obtained result. For later use it is advisable to also describe the test cases in the developer documentation or in a separate testing log.
- Non-repeatable test cases are to be avoided; they needlessly increase the costs and time of program testing. Not to mention the annoyance when we cannot repeat an erroneous run of our program, and thus the error too remains undiscovered.
- Test cases have to be made both for invalid and for valid data.
- The most possible information has to be "mined" from every test case, that is, the result of every test case has to be thoroughly examined. With this the number of necessary trials can be significantly reduced.
- When examining the results of a trial, it is equally important to determine why the program does not realize some function we expect of it, and why it also performs activities we did not assume of it.
- The testing of the program can be carried out effectively only by a person different from the writer of the program. The reason for this is that testing is not a "benevolent" activity; everyone approaches the examination of their own work by involuntarily assuming it to be good.

The methods of program testing can be divided into two groups according to whether we execute the program during testing or not. If we examine only the code of the program, then it is static (no more is said about this); if we also execute the program during testing, then we speak of dynamic testing.

Dynamic testing methods

The basic principle of dynamic testing methods is that we examine the program during operation. We can choose test cases in two ways. One possibility is the so-called black-box method, also called data-driven testing. When applying this method, the tester does not take into account the internal structure of the program – more precisely, does not regard it as the primary consideration – but chooses the test cases based on the task definition.

The goal, of course, is to carry out the most effective testing possible, that is, to find all errors in the program. This is indeed in principle possible: exhaustive input testing has to be carried out, the program has to be tried out with all possible input data. With this method, however, as we saw earlier, we can run into a quantitative obstacle.

Another possibility is the white-box method (logic-driven testing). In this method, when choosing the test cases, it is possible to also take into account the internal structure of the program.

The goal is to test the program as thoroughly as possible; a good method for this is exhaustive path testing. This means that we traverse all possible paths in the program, that is, we create as many test cases as needed to achieve this. The problem is that even for relatively small programs the number of testing paths can be very large. Think of the loops! Moreover, with this method missing paths cannot be discovered.

Since exhaustive testing is possible neither with the white-box method nor with the black-box method, we have to accept that we cannot guarantee the flawlessness of any single program. The further goal after this can be the selection, from the set of all possible test cases, of the most effective possible test-case group.

The effectiveness of testing is determined by two characteristics: the cost of testing and the proportion of discovered errors. The most effective test-case group therefore reveals the maximum number of errors at minimal cost.

Besides the black-box and white-box tests, it is also worth mentioning special tests where the goal is not the verification of correctness. Such are, e.g., the stress test (how the program copes with a large amount of data, whether it scales well) or the efficiency test (testing of execution time).

2 The concept of the data type

2.1 Basic concepts, notations

- A^* denotes the set of finite sequences in A, A^∞ the set of infinite sequences in A. Their union $A^{**} = A^* \cup A^\infty$ means the set of finite or infinite sequences in A.
- Let $R \subseteq A \times \mathbb{L}$ be a logical relation. Then the truth set of R is $\lceil R \rceil ::= R^{-1}(\{true\})$
- Let I be a finite set and let $A_i, i \in I$ be arbitrary finite or countable, nonempty sets. Then we call the set $A = \underset{i \in I}{\times} A_i$ the state space, and the sets A_i the type value sets.
- Task: we call a relation $F \subseteq A \times A$ a task.
  The above definition of the task follows naturally from the fact that we regard the task as a map on the state space, and for every point of the state space we say where one has to get to from it, if one has to get anywhere at all from it.
- Program:
  We call the relation $S \subseteq A \times A^{**}$ a program if

  1. $\mathcal{D}_S = A$ (it assigns something to every point of the state space, that is, the program does something at every point)
  2. $\forall \alpha \in \mathcal{R}_S : \alpha = red(\alpha)$ (the state changes, or if it nevertheless does not, that is a sign of abnormal operation)
  3. $\forall a \in A : \forall \alpha \in S(A) : |\alpha| \neq 0$ and $\alpha_1 = a$

  With the above definition we want to formulate the concept of "operation" in an abstract way.

2.2 Type specification

First we introduce a concept that we can use to precisely describe our requirements imposed on a type value set and the operations performable on it.

We call the triple $\mathcal{T}_\mathcal{S} = (H, I_S, \mathbb{F})$ a type specification if the following conditions hold for it:

1. H is the base set,
2. $I_S : H \to \mathbb{L}$ is the specification invariant,
3. $T_\mathcal{T} = \{(\mathcal{T}, x) | x \in \lceil I_S \rceil\}$ is the type value set,
4. $\mathbb{F} = \{F_1, F_2, ..., F_n\}$ is the specification of the type operations, where
   $\forall i \in [1..n] : F_i \subseteq A_i \times A_i$, $A_i = A_{i_1} \times ... \times A_{i_{n_i}}$ such that
   $\exists j \in [1..n_i] : A_{i_j} = T_\mathcal{T}$

With the base set and the invariant property we formulate what the set $T_\mathcal{T}$ is, with whose elements we want to deal, while with the set of tasks we describe what operations can be performed with these elements.

The type value sets appearing in the definition of the state space are all defined in such a type specification. A program can change a component of the state space only through the type operations.

2.3 Type

Let us examine how we realize the requirements described in the type specification.

We call the triple $\mathcal{T} = (\rho, I, \mathbb{S})$ a type if

1. $\rho \subseteq E^* \times T$ is the representation function (relation),
   T is the type value set,
   E is the elementary type value set

2. $I : E^* \to \mathbb{L}$ type invariant

3. $\mathbb{S} = \{S_1, S_2, ..., S_m\}$, where
   $\forall i \in [1..m] : S_i \subseteq B_i \times B_i^{**}$ is a program, $B_i = B_{i_1} \times ... \times B_{i_{m_i}}$ such that
   $\exists j \in [1..m_i] : B_{i_j} = E^*$ and $\nexists j \in [1..m_i] : B_{i_j} = T$

The first two components of the type describe the representation of the abstract type values, while the program set contains the implementation of the type operations. The elementary type value set can be the type value set of an arbitrary other type or an at-most-countable set defined in some way.

2.4 Invariant

The essence of the invariant is that we can never violate this property. For example, in the case of a set type, the invariant property that an element can occur only once in a set must not be violated.

2.5 Representation

By what types, by what method, etc., we have realized a type, we call the representation. For example, a stack type can be realized with the help of an array, but also with a linked list.

The representation is the mapping of the type value set of the type specification into the concrete type, which is given by the representation function.

2.6 Implementation

The implementation is the realization of the type operations of the type specification by the program set of the concrete type.

During implementation, when realizing the type, after the specification of the type value set, the type operations have to be defined. As in the model, in practice too the program can perform the changes of the state space only through the type operations.

2.7 In a more digestible way

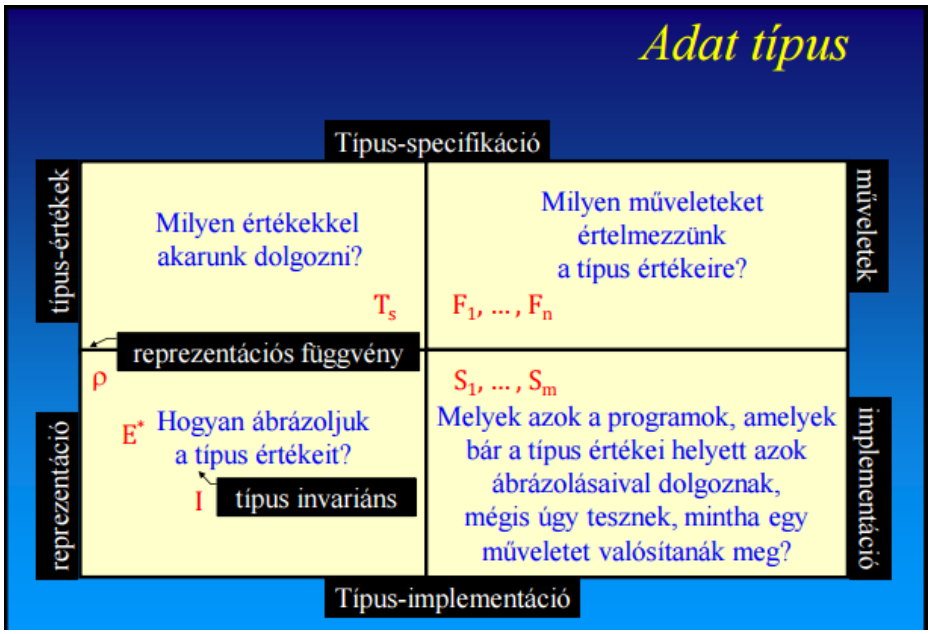


Figure 4: Data type

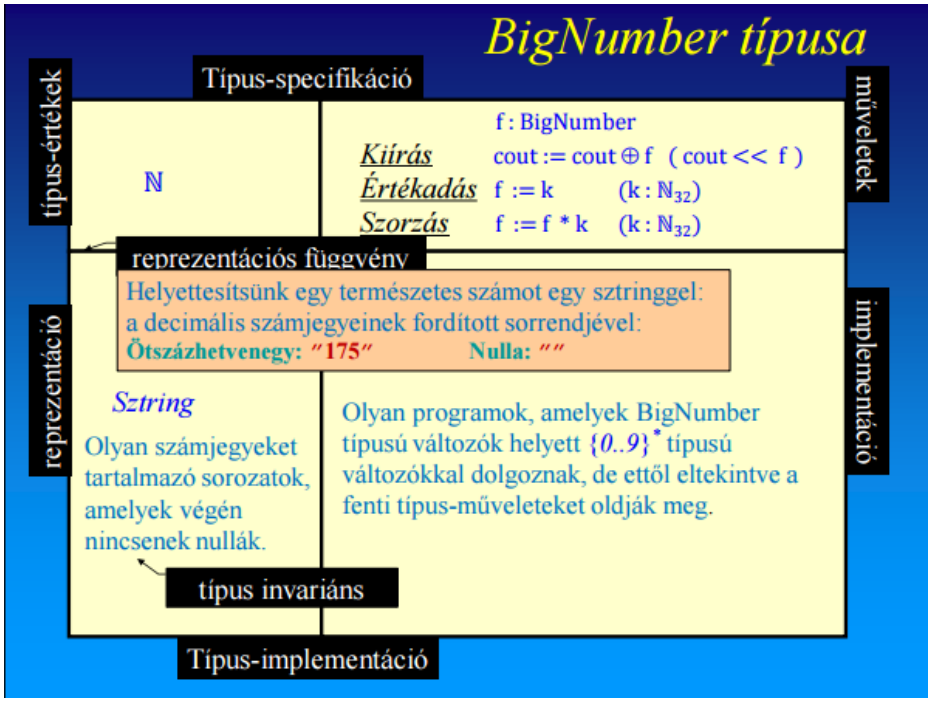


Figure 5: BigNumber example

3 The method of reduction

To solve programming tasks we use various programming patterns, so-called algorithmic patterns (programming theorems); we reduce the solution to these.

Its steps:

1. We guess the algorithmic pattern solving the task.
2. We specify the task with the notations of the algorithmic pattern.

3. We give the differences between the algorithmic pattern and the task:
   - interval bounds: concrete value or expression (e.g. $[1..\frac{n}{2}]$), their type instead of $\mathbb{Z}$ can be some part of it (e.g. $\mathbb{N}$)
   - the concrete counterparts of $\beta : [m..n] \to \mathbb{L}$ and/or $f : [m..n] \to H$
   - the counterpart of H with the necessary operation
     - instead of $(H, >)$ e.g. $(\mathbb{Z}, >)$ or $(\mathbb{Z}, <)$
     - instead of $(H, +)$ e.g. $(\mathbb{Z}, +)$ or $(\mathbb{R}, *)$
   - the renaming of the variables
4. Taking the differences into account, we produce from the algorithm of the theorem the algorithm solving the concrete task.

4 The enumerator, the specification of the enumerator type

The collection (container, collection, iterated) is a data (object) suitable for storing some elements.

- Such are the values of composite-structured, but especially iterated-structured types: set, sequence (stack, queue, file), tree, graph
- But there are also so-called virtual collections: e.g. the elements of an interval of integers, or the prime divisors of a natural number

By the processing of a collection we mean the processing of the elements in it.

- Find the largest element of the set!
- How many negative numbers are in a number sequence?
- Select the values placed in the leaves of a tree!
- Traverse every second element of the interval [m .. n] backwards!
- Sum the prime divisors of the natural number n!

We standardize the enumeration (traversal) of the elements to be processed with the following operations:

- First() : Positions onto the first element of the enumeration, that is, starts the enumeration
- Next() : Positions onto the next element of the started enumeration
- End() : Indicates if we have reached the end of the enumeration
- Current() : Returns the current element of the enumeration

An enumeration has different states (ready to start, in progress, finished), and the operations are defined only in certain states (elsewhere their effect is undefined). The processing algorithm guarantees that the enumerator operations are always executed in the appropriate state.

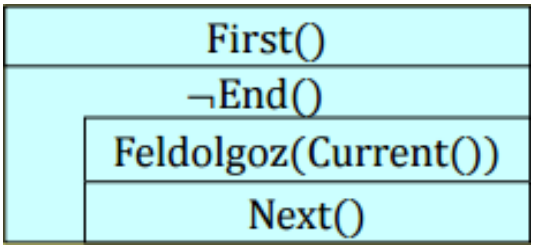


Figure 6: The algorithm of enumeration

The enumeration is never done by the collection to be enumerated, but by a separate enumerator object.

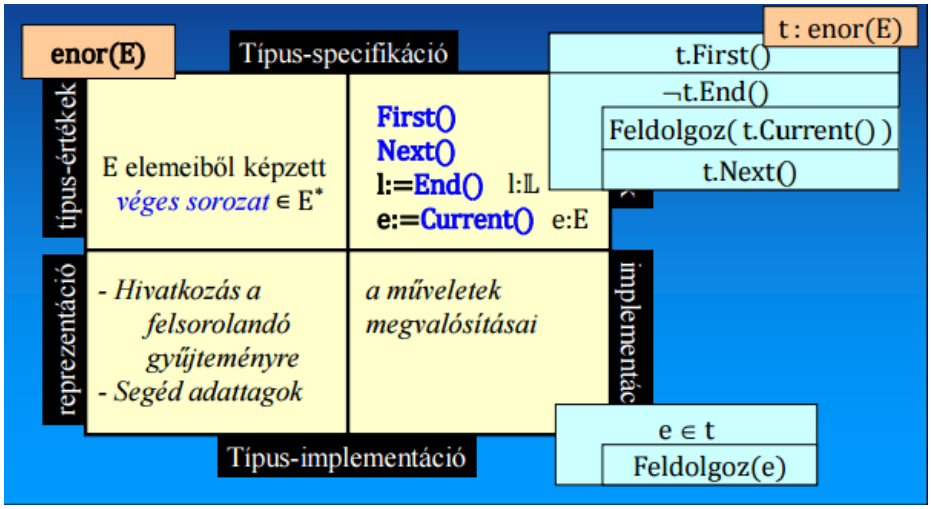


Figure 7: The enumerator object and its type

5 Algorithmic patterns formulated on enumerators

Programozási tételek felsorolókra

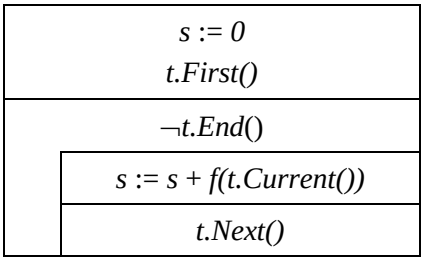
Összegzés

Feladat: Adott egy *E*-beli elemeket felsoroló *t* objektum és egy $f:E\to H$ függvény. A *H* halmazon értelmezzük az összeadás asszociatív, baloldali nullelemes műveletét. Határozzuk meg a függvénynek a *t* elemeihez rendelt értékeinek összegét! (Üres felsorolás esetén az összeg értéke definíció szerint a nullelem: *0*).

Specifikáció:

$A = (t{:}enor(E), s{:}H)$
$Ef = (t=t')$
$Uf = (s = \sum_{e \in t'} f(e))$

Algoritmus:



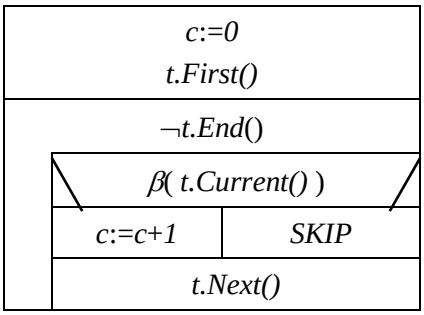
Számlálás

Feladat: Adott egy *E*-beli elemeket felsoroló *t* objektum és egy $\beta:E\to\mathbb{L}$ feltétel. A felsoroló objektum hány elemére teljesül a feltétel?

Specifikáció:

$A = (t{:}enor(E), c{:}\mathbb{N})$
$Ef = (t=t')$
$Uf = (c = \sum_{\substack{e \in t' \\ \beta(e)}} 1)$

Algoritmus:



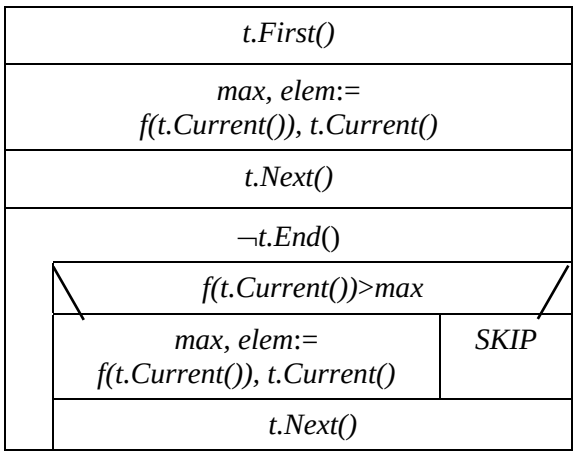
Maximum kiválasztás

Feladat: Adott egy *E*-beli elemeket felsoroló *t* objektum és egy $f:E\to H$ függvény. A *H* halmazon definiáltunk egy teljes rendezési relációt. Feltesszük, hogy *t* nem üres. Hol veszi fel az *f* függvény a *t* elemein a maximális értékét?

Specifikáció:

$A = (t{:}enor(E), max{:}H, elem{:}E)$
$Ef = (t=t' \wedge |t|>0)$
$Uf = ((max, elem) = \mathbf{max}_{e \in t'} f(e))$

Algoritmus:



Kiválasztás

Feladat: Adott egy *E*-beli elemeket felsoroló *t* objektum és egy β:$E \to \mathbb{L}$ feltétel. Keressük a *t* bejárása során az első olyan elemi értéket, amely kielégíti a β:$E \to \mathbb{L}$ feltételt, ha tudjuk, hogy biztosan van ilyen.

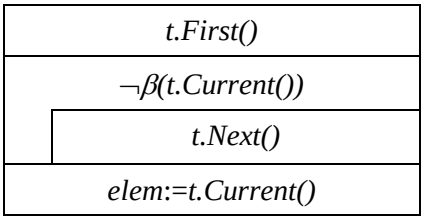
Specifikáció:

$A = (t{:}enor(E), elem{:}E)$

$Ef = (t=t' \wedge \exists i \in [1..|t|]: \beta(t_i))$

$Uf = ((elem,t) = \underset{elem \in t'}{\textbf{select}} \beta(elem))$

Algoritmus:



Lineáris keresés

Feladat: Adott egy *E*-beli elemeket felsoroló *t* objektum és egy β:$E \to \mathbb{L}$ feltétel. Keressük a *t* bejárása során az első olyan elemi értéket, amely kielégíti a β:$E \to \mathbb{L}$ feltételt.

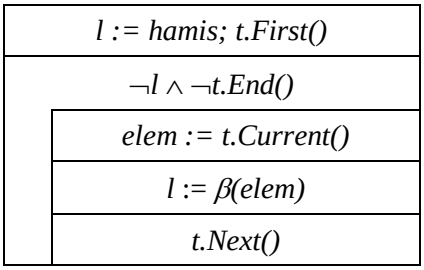
Specifikáció:

$A = (t{:}enor(E), l{:}\mathbb{L}, elem{:}E)$

$Ef = (t=t')$

$Uf = ((l,elem,t) = \underset{e \in t'}{\textbf{search}} \beta(e))$

Algoritmus:



Feltételes maximumkeresés

Feladat: Adott egy *E*-beli elemeket felsoroló *t* objektum, egy β:$E \to \mathbb{L}$ feltétel és egy $f{:}E \to H$ függvény. A *H* halmazon definiáltunk egy teljes rendezési relációt. Határozzuk meg *t* azon elemeihez rendelt *f* szerinti értékek között a legnagyobbat, amelyek kielégítik a β feltételt.

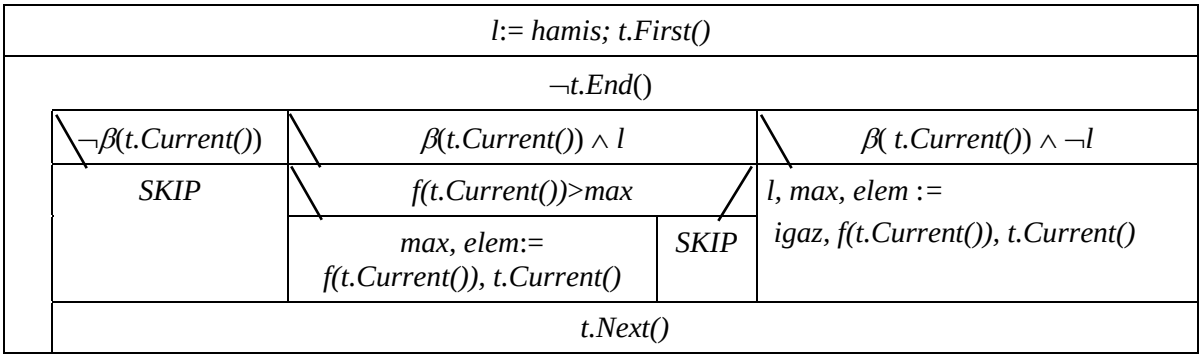
Specifikáció:

$A = (t{:}enor(E), l{:}\mathbb{L}, max{:}H, elem{:}E)$

$Ef = (t=t')$

$Uf = ((l, max, elem) = \underset{\substack{e \in t' \\ \beta(e)}}{\textbf{max}} f(e))$

Algoritmus:



6 Enumerators of well-known collections

6.1 Interval

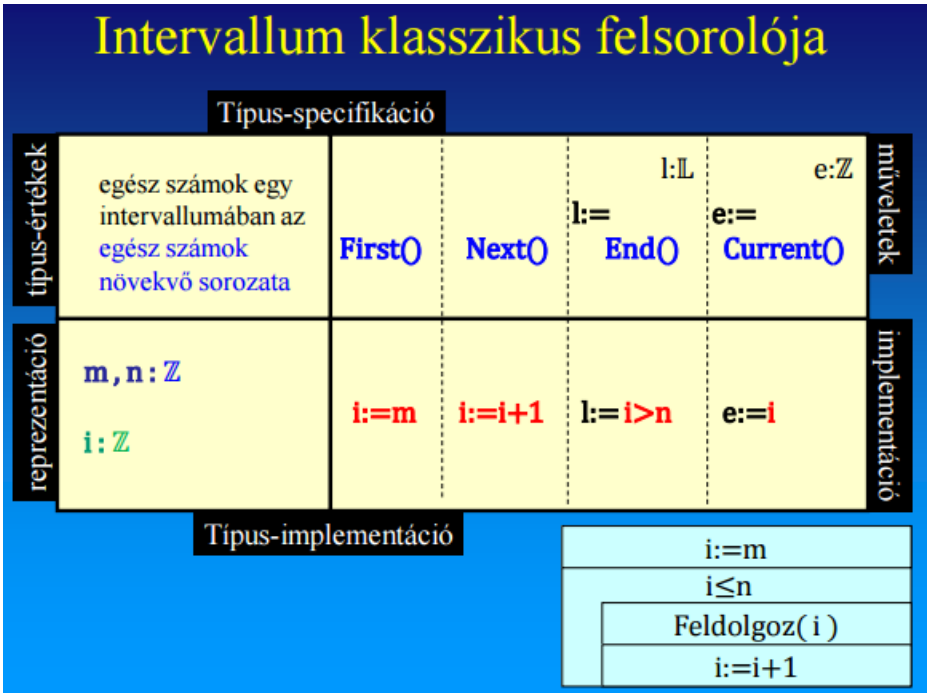


Figure 8: Enumerator of an interval

6.2 Array

Here we present the enumerator of two different array types: the one-dimensional (vector) and the two-dimensional array (matrix).

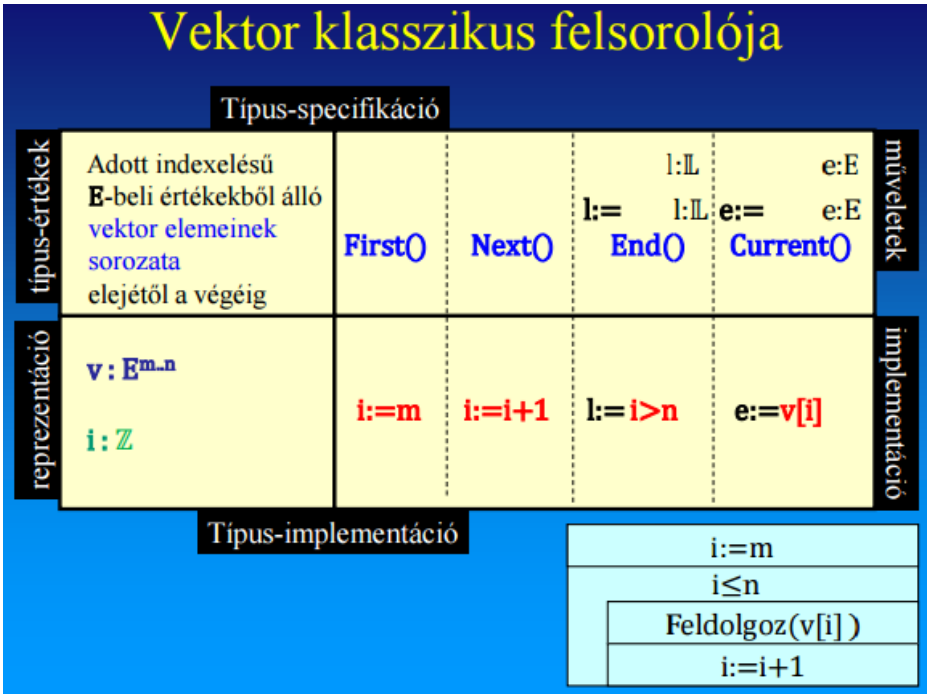


Figure 9: Enumerator of a vector

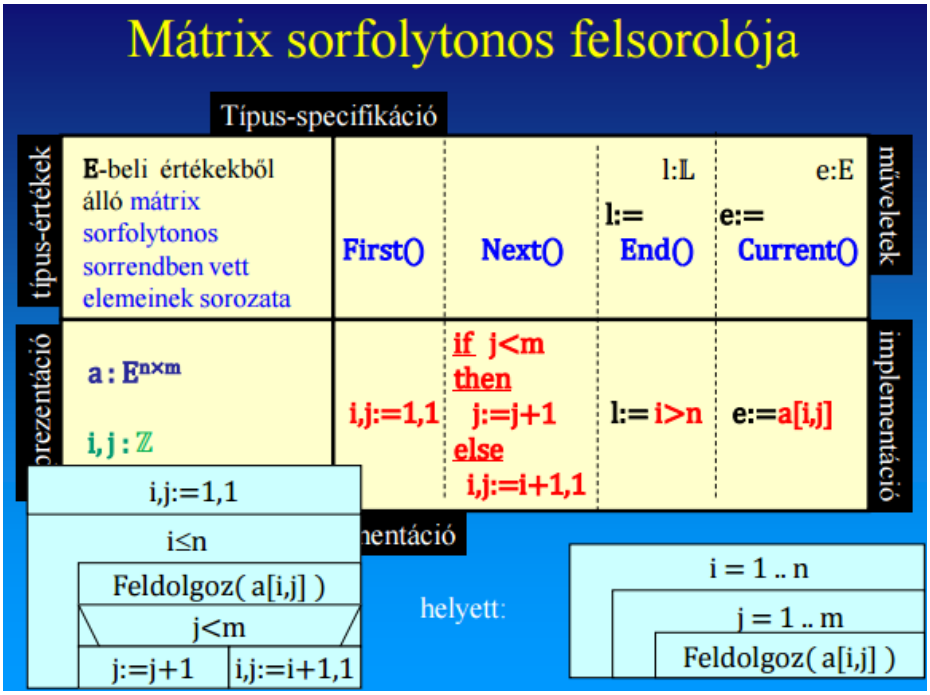


Figure 10: Row-major enumerator of a matrix

Remark: the enumeration can also be done differently, for example for a vector we can perform the enumeration backwards, starting from the end of the array, or for a matrix we can apply e.g. column-major traversal.

6.3 Sequence

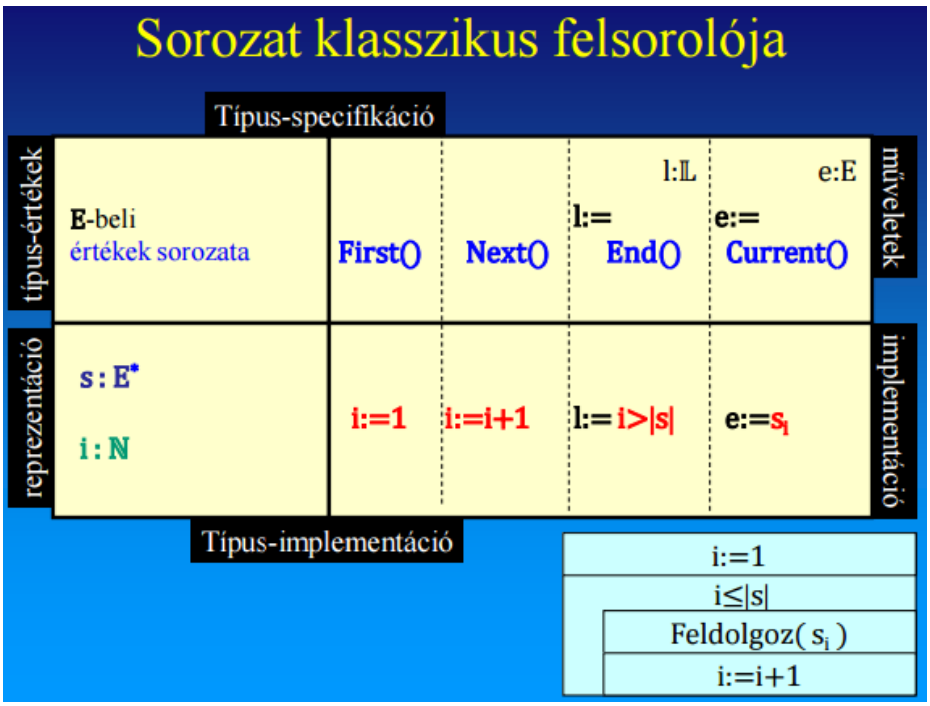


Figure 11: Enumerator of a sequence

6.4 Set

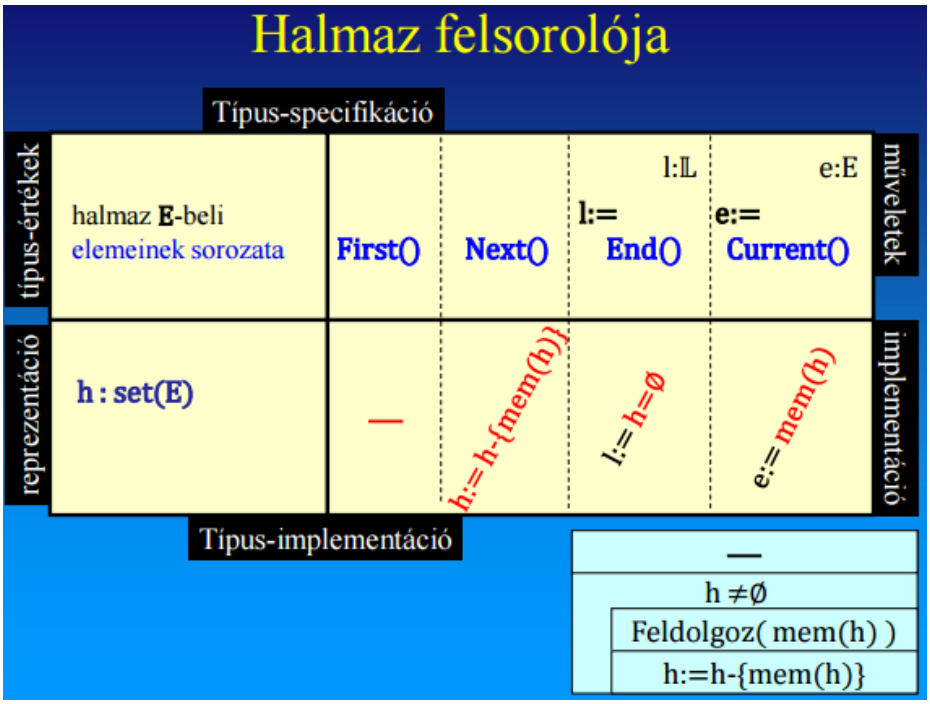


Figure 12: Enumerator of a set

6.5 Sequential input file

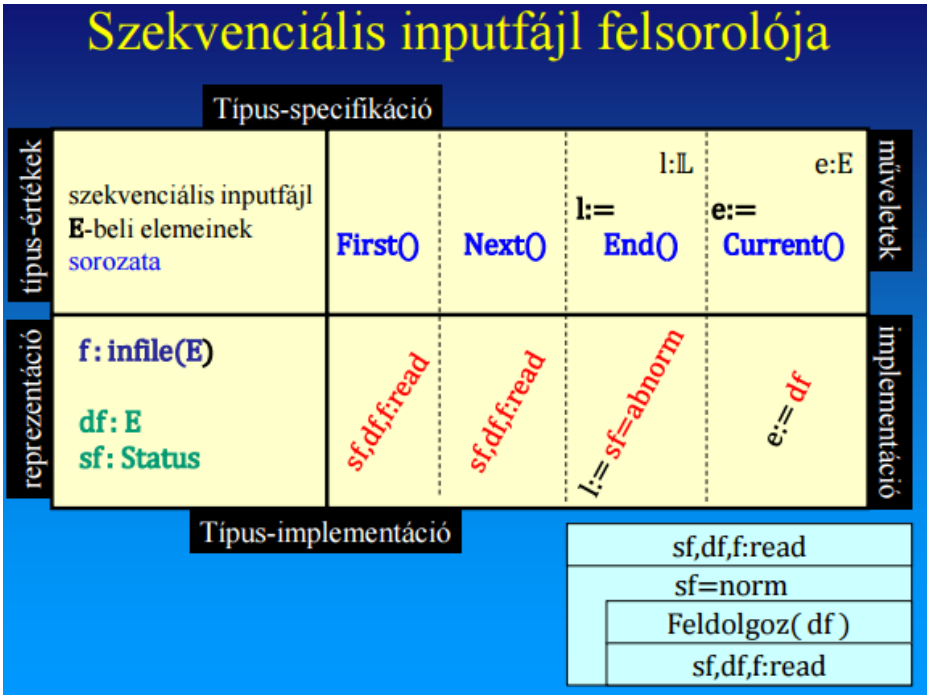


Figure 13: Enumerator of a sequential input file

7 Testing programs created with algorithmic patterns

There are three kinds of testing strategy, the following.

1. Black box: test cases written based on the (specification of the) task.

(a) Test cases written with test data satisfying the precondition (valid), or violating it (invalid).

(b) Examination of test cases generated based on the postcondition (?).

2. White box: test cases written based on the code.

   (a) Trying out every statement of the algorithm

   (b) Trying out every control node (branching, loop) of the algorithm

3. Grey box: test cases checking the operation of the algorithm projected by the executable specification.

   (a) If, moreover, the executable specification originates from an algorithm pattern, then the usual test cases of the algorithm pattern have to be examined.

The algorithmic patterns are algorithm patterns, so here we will deal with their test cases.

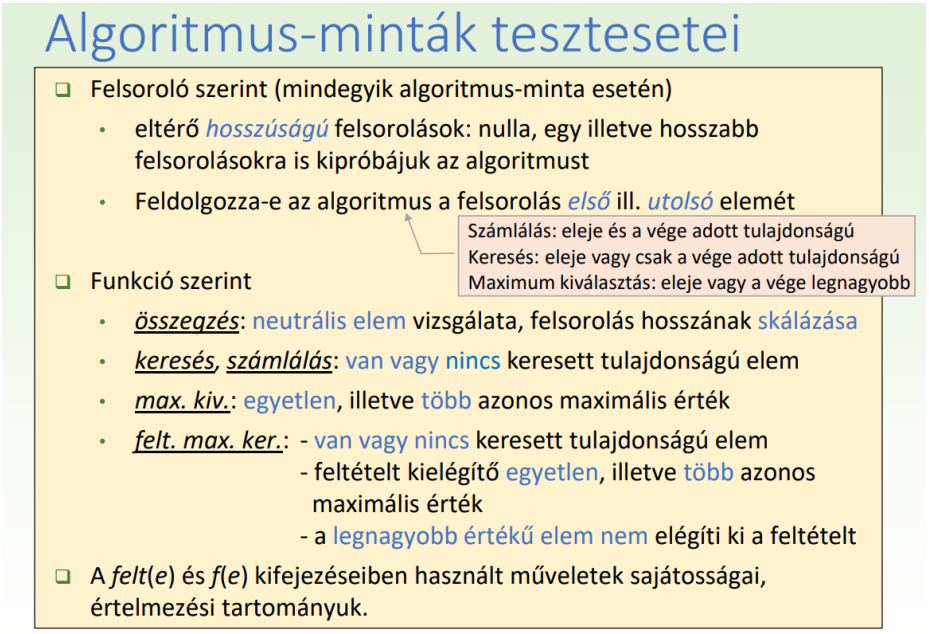


Figure 14: Test cases of algorithm patterns

Example task built on an algorithmic pattern (programming pattern):

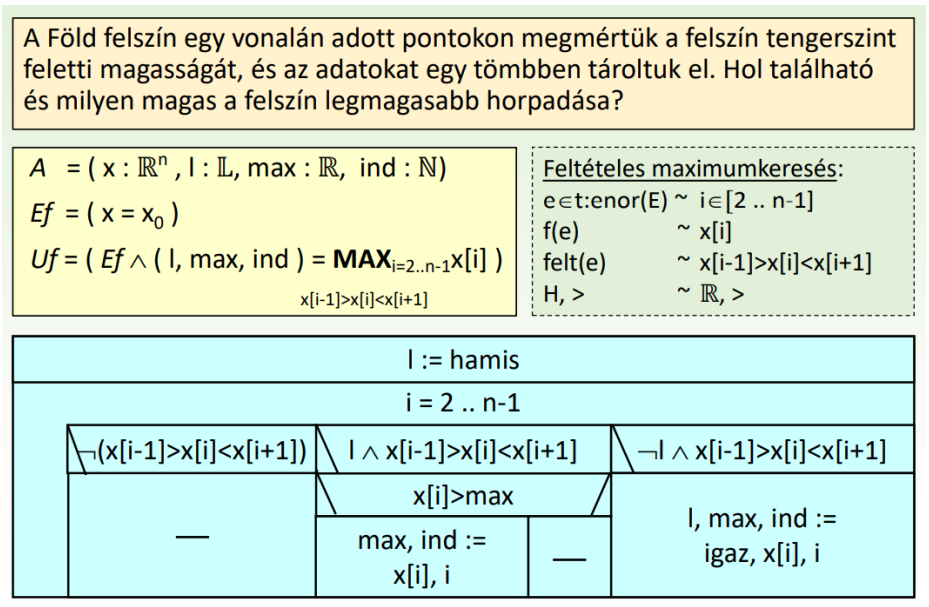


Figure 15: Task built on an algorithmic pattern

Feltételes maximum keresés tesztesetei			
felsoroló szerint	hossza: 0	x = < > , < 1.0, 2.0>	→ l = hamis
	hossza: 1	x = < 2.1, 1.0, 2.4 >	→ l = igaz, max = 1.0, ind = 2
	hossza: több	x = < 1.0, 2.0, 1.5, 4.0, 2.0 >	→ l = igaz, max = 1.5, ind = 3
	eleje	x = < 3.0, 2.5, 3.0, 2.0, 3.0 >	→ l = igaz, max = 2.5, ind = 2
	vége	x = < 3.0, 2.0, 3.0, 2.5, 3.0 >	→ l = igaz, max = 2.5, ind = 4
tétel szerint	nincs	x = < 1.0, 2.0, 3.0, 4.0, 5.0 >	→ l = hamis
	van	x = < 1.0, 2.0, 1.0, 4.0, 2.0 >	→ l = igaz, max = 1.0, ind = 3
	egy maximum	x = < 3.0, 1.5, 3.0, 2.5, 3.0 >	→ l = igaz, max = 2.5, ind = 4
	több maximum	x = < 3.0, 2.0, 3.0, 2.0, 3.0 >	→ l = igaz, max = 2.0, ind = 2,4

Figure 16: Example test cases